MLIR Part 3 - Affine Dialect and OpenMP

Stephen Diehl

The affine dialect in MLIR provides abstractions for expressing loops and array accesses that are amenable to parallel execution. Unlike the `scf` dialect we've seen before, the affine dialect enables advanced loop transformations and automatic parallelization. It's esesntially "first class loops" which is something we always used to layer on top of our LLVM.

The affine dialect many advantages, including automatic parallelization opportunities, built-in vectorization capabilities, sophisticated loop optimizations, a natural expression of array computations, and integration with OpenMP.

Polyhedral Optimization Model

The polyhedral model is a mathematical framework that represents loop nests and their computations as geometric objects in multi-dimensional spaces.

At its core, the polyhedral model represents the execution of statements inside loop nests as points in a multi-dimensional space. Each point corresponds to a specific instance of a statement in the execution, defined by its iteration coordinates. This representation enables program analysis and transformation techniques by converting complex loop optimization problems into geometric operations on polyhedra.

The key components of the polyhedral model include iteration domains, access relations, and scheduling

functions. Iteration domains define the set of all iterations executed by a loop nest, represented as a set of integer points bounded by affine constraints. Access relations map iteration points to the memory locations accessed by each statement. Scheduling functions determine the order of execution for iteration points, enabling transformations that preserve program semantics while improving performance characteristics.

MLIR provides the `affine` that incorporates polyhedral concepts as first-class citizens. This hybrid design optimizes the representation, analysis, and transformation of high-level dataflow graphs and target-specific code for high-performance data-parallel systems.

There are three core concepts we have to understand:

Affine maps: Multi-dimensional quasi-linear functions that map dimensions and symbols to results. For example, `(d0, d1, d2, s0) → (d0 + d1, s0 * d2)` represents a two-dimensional affine map that maps dimension arguments `d0`, `d1`, `d2` and symbol argument `s0` to two results.

Integer sets: Constraints on dimensions and symbols represented as affine inequalities and equalities. For instance, `(i)[N, M] : (i >= 0, -i + N >= 0, N - 5 == 0, -i + M + 1 >= 0)` represents the set of values `i` such that `0 <= i < N`, `N = 5`, and `i <= M + 1`.

Affine operations: Operations such as `affine.for`, `affine.if`, and `affine.parallel` that leverage affine maps and integer sets to represent loop nests and conditionals with affine constraints.

Rather than front-load the theory, let's just look at some examples.

Matrix Multiplication

The classic matrix product of two matrices A_{ij} and B_{jk} is:

$$C_{ik} = (AB)_{ik} = \sum_{j=1}^{N} A_{ij} B_{jk}$$

Expressing this in MLIR as an explicit loop nest would look like this:

```
func.func @matmul(%A: memref<?x?xf32>, %B: memref<?x?xf32>, %C: memref<?x?xf32>) {
  %c0 = arith.constant 0 : index
  %c1 = arith.constant 1 : index

  %M = memref.dim %A, %c0 : memref<?x?xf32>
  %N = memref.dim %B, %c1 : memref<?x?xf32>
  %K = memref.dim %A, %c1 : memref<?x?xf32>

  // Sequential implementation
  affine.for %i = 0 to %M {
    affine.for %j = 0 to %N {
      affine.for %k = 0 to %K {
        %a = affine.load %A[%i, %k] : memref<?x?xf32>
        %b = affine.load %B[%k, %j] : memref<?x?xf32>
        %c = affine.load %C[%i, %j] : memref<?x?xf32>
        %prod = arith.mulf %a, %b : f32
        %sum = arith.addf %c, %prod : f32
```

```
        affine.store %sum, %C[%i, %j] : memref<?x?xf32>
      }
    }
  }
  return
}
```

Notice that the two loops over `%i` and `%j` have no dependencies between different iterations of these loops. Using the `-affine-parallelize` pass, this can be transformed into:

```
func.func @matmul_parallel(%A: memref<?x?xf32>, %B: memref<?x?xf32>, %C: memref<?
x?xf32>) {
  %c0 = arith.constant 0 : index
  %c1 = arith.constant 1 : index

  %M = memref.dim %A, %c0 : memref<?x?xf32>
  %N = memref.dim %B, %c1 : memref<?x?xf32>
  %K = memref.dim %A, %c1 : memref<?x?xf32>

  // Parallel implementation
  affine.parallel (%i, %j) = (0, 0) to (%M, %N) {
    affine.for %k = 0 to %K {
      %a = affine.load %A[%i, %k] : memref<?x?xf32>
      %b = affine.load %B[%k, %j] : memref<?x?xf32>
      %c = affine.load %C[%i, %j] : memref<?x?xf32>
      %prod = arith.mulf %a, %b : f32
      %sum = arith.addf %c, %prod : f32
      affine.store %sum, %C[%i, %j] : memref<?x?xf32>
    }
  }
  return
}
```

The `affine.parallel` operation represents a way to express that multiple iterations of a loop can be executed independently and potentially simultaneously. Think of it like saying "these calculations can happen in any order, or all at once" - similar to how you might divide up work among multiple workers who don't need to communicate with each other.

More precisely, `affine.parallel` defines a set of nested loops where:

Each iteration is independent of all other iterations

The iterations can be executed in any order or simultaneously

The operation produces results through reductions (like summing up values) where the order of combining results doesn't matter

Technically, it creates a "hyper-rectangular parallel band," which means it can represent multiple nested parallel

loops, each characterized by its own induction variables (loop counters), lower and upper bounds, step sizes (indicating how much to increment each iteration), and reduction operations (defining how to combine results from different iterations).

The body of the parallel region must end with `affine.yield` which specifies how to combine results from different iterations using operations like addition or multiplication. For loops that execute zero times, the result is the identity value for that operation (0 for addition, 1 for multiplication).

Affine Maps (`affine.apply`)

Affine maps are a key component of the affine dialect, representing multi-dimensional affine transformations. They provides us a way to describe the mapping of loop indices to memory access patterns, which is essential for optimizing memory layouts and access patterns in high-performance computing.

Let's look at a comprehensive example that demonstrates various uses of `affine.apply`:

```
// Define some reusable affine maps
#tile_map = affine_map<(d0) -> (d0 floordiv 32)>
#offset_map = affine_map<(d0)[s0] -> (d0 + s0)>
#complex_map = affine_map<(d0, d1)[s0] -> (d0 * 2 + d1 floordiv 4 + s0)>

func.func @affine_apply_examples() {
  // Create some test indices
  %c0 = arith.constant 0 : index
  %c42 = arith.constant 42 : index
  %c128 = arith.constant 128 : index

  // Example 1: Simple tiling calculation
  // Calculate which tile a given index belongs to (tile size = 32)
  %tile_idx = affine.apply #tile_map(%c42)

  // Example 2: Inline affine map for offset calculation
  %offset = affine.apply affine_map<(i) -> (i + 10)>(%c42)

  // Example 3: Using a symbol parameter
  %shifted = affine.apply #offset_map(%c42)[%c128]

  // Example 4: Multiple dimensions and a symbol
  %complex = affine.apply #complex_map(%c42, %c128)[%c0]

  // Example 5: Composition of affine applies
  %temp = affine.apply affine_map<(i) -> (i * 2)>(%c42)
  %final = affine.apply affine_map<(i) -> (i + 5)>(%temp)

  return
}
```

```
// Example showing how affine.apply can be used in a practical loop context
func.func @tiled_loop(%arg0: memref<256xf32>) {
  %c0 = arith.constant 0 : index
  %c256 = arith.constant 256 : index
  %c32 = arith.constant 32 : index

  // Outer loop iterates over tiles
  affine.for %i = 0 to 256 step 32 {
    // Inner loop processes elements within the tile
    affine.for %j = 0 to 32 {
      %idx = affine.apply affine_map<(d0, d1) -> (d0 + d1)>(%i, %j)
      %val = memref.load %arg0[%idx] : memref<256xf32>
      // Process value...
      memref.store %val, %arg0[%idx] : memref<256xf32>
    }
  }
  return
}
```

The `tiled_loop` function shows how `affine.apply` can be used in a real-world scenario to implement tiled processing of an array. The outer loop steps by the tile size, and `affine.apply` is used to calculate the bounds for the inner loop that processes each tile.

Some key points about affine maps include that `floordiv` performs integer division rounding down, and the syntax `(d0)[s0]` indicates one dimension parameter `d0` and one symbol parameter `s0`. Additionally, affine maps can only contain addition, subtraction, multiplication by constants, division by constants, modulo by constants, and floordiv/ceildiv by constants. The result type is always `index`.

Affine Optimization Passes

The MLIR optimizer provides a rich set of passes specifically designed for the affine dialect. These passes can automatically transform code to improve performance through various optimization techniques. Let's explore some of the most important ones:

Loop invariant code motion: This transformation moves loop invariant code (code that does not depend on the loop index) outside the loop. This can help expose parallelism by moving code that is not dependent on the loop index to the outer loop.

```
// Before
func.func @before(%A: memref<10xf32>, %B: memref<10xf32>) {
  %c0 = arith.constant 0 : index
  %c10 = arith.constant 10 : index
  %c1 = arith.constant 1 : index

  affine.for %i = 0 to 10 {
```

```
    %x = arith.constant 42.0 : f32  // This is loop invariant!
    %v = memref.load %A[%i] : memref<10xf32>
    %sum = arith.addf %v, %x : f32
    memref.store %sum, %B[%i] : memref<10xf32>
  }
  return
}

// After
func.func @after(%A: memref<10xf32>, %B: memref<10xf32>) {
  %c0 = arith.constant 0 : index
  %c10 = arith.constant 10 : index
  %c1 = arith.constant 1 : index

  %x = arith.constant 42.0 : f32  // Moved outside the loop
  affine.for %i = 0 to 10 {
    %v = memref.load %A[%i] : memref<10xf32>
    %sum = arith.addf %v, %x : f32
    memref.store %sum, %B[%i] : memref<10xf32>
  }
  return
}
```

Loop skewing: This transformation changes the iteration space by shifting iterations based on the values of other loop indices. This can help expose parallelism by transforming dependencies.

```
// Before - Dependencies prevent parallelization
func.func @before(%A: memref<10x10xf32>) {
  affine.for %i = 1 to 10 {
    affine.for %j = 1 to 10 {
      %v1 = affine.load %A[%i-1, %j] : memref<10x10xf32>
      %v2 = affine.load %A[%i, %j-1] : memref<10x10xf32>
      %sum = arith.addf %v1, %v2 : f32
      affine.store %sum, %A[%i, %j] : memref<10x10xf32>
    }
  }
}

// After skewing - Inner loop can now be parallelized
func.func @after(%A: memref<10x10xf32>) {
  affine.for %i = 1 to 10 {
    affine.for %j = %i to %i + 9 {
      %i_new = affine.apply affine_map<(d0, d1) -> (d0)>(%i, %j)
      %j_new = affine.apply affine_map<(d0, d1) -> (d1 - d0)>(%i, %j)
      %v1 = affine.load %A[%i_new-1, %j_new] : memref<10x10xf32>
```

```
      %v2 = affine.load %A[%i_new, %j_new-1] : memref<10x10xf32>
      %sum = arith.addf %v1, %v2 : f32
      affine.store %sum, %A[%i_new, %j_new] : memref<10x10xf32>
    }
  }
}
```

Loop interchange: This transformation changes the nesting order of loops, which can improve cache locality and expose parallelism.

```
// Before - Column-major access pattern
func.func @before(%A: memref<16x16xf32>, %B: memref<16x16xf32>) {
  affine.for %i = 0 to 16 {
    affine.for %j = 0 to 16 {
      %v = affine.load %A[%j, %i] : memref<16x16xf32>
      affine.store %v, %B[%j, %i] : memref<16x16xf32>
    }
  }
}

// After - Row-major access pattern (better locality)
func.func @after(%A: memref<16x16xf32>, %B: memref<16x16xf32>) {
  affine.for %j = 0 to 16 {
    affine.for %i = 0 to 16 {
      %v = affine.load %A[%j, %i] : memref<16x16xf32>
      affine.store %v, %B[%j, %i] : memref<16x16xf32>
    }
  }
}
```

Loop fusion: This transformation combines adjacent loops to improve locality and reduce loop overhead.

```
// Before fusion
func.func @before(%A: memref<10xf32>, %B: memref<10xf32>, %C: memref<10xf32>) {
  affine.for %i = 0 to 10 {
    %v1 = affine.load %A[%i] : memref<10xf32>
    %v2 = arith.mulf %v1, %v1 : f32
    affine.store %v2, %B[%i] : memref<10xf32>
  }

  affine.for %i = 0 to 10 {
    %v3 = affine.load %B[%i] : memref<10xf32>
    %v4 = arith.addf %v3, %v3 : f32
    affine.store %v4, %C[%i] : memref<10xf32>
  }
}
```

```
// After fusion
func.func @after(%A: memref<10xf32>, %B: memref<10xf32>, %C: memref<10xf32>) {
  affine.for %i = 0 to 10 {
    %v1 = affine.load %A[%i] : memref<10xf32>
    %v2 = arith.mulf %v1, %v1 : f32
    affine.store %v2, %B[%i] : memref<10xf32>
    %v3 = arith.addf %v2, %v2 : f32
    affine.store %v3, %C[%i] : memref<10xf32>
  }
}
```

Loop tiling: This transformation breaks a loop into smaller chunks to improve cache locality.

```
// Before tiling
func.func @before(%A: memref<32x32xf32>) {
  affine.for %i = 0 to 32 {
    affine.for %j = 0 to 32 {
      %v = affine.load %A[%i, %j] : memref<32x32xf32>
      // ... computation ...
      affine.store %v, %A[%i, %j] : memref<32x32xf32>
    }
  }
}

// After tiling (tile size 8x8)
func.func @after(%A: memref<32x32xf32>) {
  affine.for %ti = 0 to 32 step 8 {
    affine.for %tj = 0 to 32 step 8 {
      affine.for %i = #map(%ti) to #map(%ti + 8) {
        affine.for %j = #map(%tj) to #map(%tj + 8) {
          %v = affine.load %A[%i, %j] : memref<32x32xf32>
          // ... computation ...
          affine.store %v, %A[%i, %j] : memref<32x32xf32>
        }
      }
    }
  }
}
```

These optimizations are available in the `mlir-opt` tool:

`-affine-loop-coalescing` - Merge consecutive loops into a single loop.

`-affine-loop-fusion` - Merge loops with the same bounds and steps.

`-affine-loop-invariant-code-motion` - Move loop invariant code outside the loop.

`-affine-loop-normalize` - Normalize loop bounds and steps.

`-affine-loop-tile` - Tile loops.

`-affine-loop-unroll` - Unroll loops.

`-affine-loop-unroll-jam` - Unroll loops and jam them together.

`-affine-parallelize` - Parallelize loops.

`-affine-pipeline-data-transfer` - Pipeline non-blocking data transfers between explicitly managed levels of the memory hierarchy

`-affine-scalrep` - Replace affine memref accesses by scalars by forwarding stores to loads and eliminating redundant loads

`-affine-simplify-structures` - Simplify affine expressions in maps/sets and normalize memrefs

`-affine-super-vectorize` - Vectorize to a target independent n-D vector abstraction

There is also the `-convert-affine-for-to-gpu`, which we'll use later, which converts affine loops to GPU kernels which can target multiple GPU architectures.

Convolution Kernels

Now that we understand the basic optimizations available in the affine dialect, let's look at a more practical example: implementing a 2D convolution. This operation is fundamental in image processing and deep learning, where it's used for feature extraction and pattern recognition.

The mathematical formula for 2D convolution is:

$$(A * K)_{ij} = \sum_u \sum_v K_{uv} A_{(i+u)(j+v)}$$

```
module {
  func.func @conv_2d(%input: memref<128x128xf32>, %filter: memref<16x16xf32>,
%output: memref<113x113xf32>) {
    // Loop over the output matrix dimensions (113x113)
    affine.for %i = 0 to 113 {
      affine.for %j = 0 to 113 {
        // Use affine.parallel to accumulate values into %acc using iter_args
        %zero = arith.constant 0.0 : f32
        %acc = affine.for %fi = 0 to 16 iter_args(%acc = %zero) -> (f32) {
          %acc_inner = affine.for %fj = 0 to 16 iter_args(%acc_inner = %acc) ->
(f32) {
            // Load filter value
            %filter_val = affine.load %filter[%fi, %fj] : memref<16x16xf32>

            // Load corresponding input value from the input matrix
            %input_val = affine.load %input[%i + %fi, %j + %fj] :
memref<128x128xf32>
```

```
            // Multiply input value with filter value
            %prod = arith.mulf %input_val, %filter_val : f32

            // Add product to the accumulator
            %new_acc = arith.addf %acc_inner, %prod : f32
            affine.yield %new_acc : f32
          }
          affine.yield %acc_inner : f32
        }

        // Store the accumulated result in the output matrix
        affine.store %acc, %output[%i, %j] : memref<113x113xf32>
      }
    }
    return
  }
}
```

To properly optimize and parallelize affine operations, we need a specific sequence of passes:

```
mlir-opt conv_2d.mlir \
  --affine-loop-normalize \
  --affine-parallelize \
  -lower-affine \
  --convert-scf-to-cf \
  --convert-cf-to-llvm \
  --convert-math-to-llvm \
  --convert-arith-to-llvm \
  --finalize-memref-to-llvm \
  --convert-func-to-llvm \
  --reconcile-unrealized-casts \
  -o conv_2d_opt.mlir

# Translate to LLVM IR
mlir-translate conv_2d_opt.mlir -mlir-to-llvmir -o conv_2d.ll

# Compile to object file
llc -filetype=obj --relocation-model=pic conv_2d.ll -o conv_2d.o

# Compile to shared library
clang -shared -fopenmp -o libconv2d.dylib conv_2d.o
```

Here the `-lower-affine` pass lowers the affine dialect to combinations of the `scf` and `arith` dialects.

To use our parallelized code from Python, we've created a helper module to handle 2D array conversions between NumPy and MLIR MemRef descriptors. The module provides utilities for converting NumPy arrays and

running the compiled MLIR code:

```
import ctypes
import numpy as np
from ctypes import c_void_p, c_longlong, Structure


class MemRef2DDescriptor(Structure):
    """Structure matching MLIR's 2D MemRef descriptor"""

    _fields_ = [
        ("allocated", c_void_p),  # Allocated pointer
        ("aligned", c_void_p),  # Aligned pointer
        ("offset", c_longlong),  # Offset in elements
        ("shape", c_longlong * 2),  # Array shape (2D)
        ("stride", c_longlong * 2),  # Strides in elements
    ]


def numpy_to_memref2d(arr):
    """Convert a 2D NumPy array to a MemRef descriptor"""
    if not arr.flags["C_CONTIGUOUS"]:
        arr = np.ascontiguousarray(arr)

    desc = MemRef2DDescriptor()
    desc.allocated = arr.ctypes.data_as(c_void_p)
    desc.aligned = desc.allocated
    desc.offset = 0
    desc.shape[0] = arr.shape[0]
    desc.shape[1] = arr.shape[1]
    desc.stride[0] = arr.strides[0] // arr.itemsize
    desc.stride[1] = arr.strides[1] // arr.itemsize

    return desc


def run_conv2d():
    """Run 2D convolution using MLIR compiled module"""
    # Create input arrays
    input_matrix = np.ones((10, 10), dtype=np.float32)
    conv_filter = np.arange(9, dtype=np.float32).reshape(3, 3)
    result = np.zeros((8, 8), dtype=np.float32)

    # Load compiled module
```

```
    module = ctypes.CDLL("./libconv2d.dylib")

    # Prepare MemRef descriptors
    input_memref = numpy_to_memref2d(input_matrix)
    filter_memref = numpy_to_memref2d(conv_filter)
    result_memref = numpy_to_memref2d(result)

    # Set function argument types
    module.conv_2d.argtypes = [
        *[type(x) for x in input_memref._fields_],
        *[type(x) for x in filter_memref._fields_],
        *[type(x) for x in result_memref._fields_],
    ]

    # Call the function
    module.conv_2d(
        *[getattr(input_memref, field[0]) for field in input_memref._fields_],
        *[getattr(filter_memref, field[0]) for field in filter_memref._fields_],
        *[getattr(result_memref, field[0]) for field in result_memref._fields_]
    )

    return result


if __name__ == "__main__":
    result = run_conv2d()
    print(result)
```

OpenMP

While the affine dialect provides abstractions for parallel execution, sometimes we want more direct control over parallelization. This is where OpenMP comes in. OpenMP is a directive-based model that provides explicit control over parallel execution, making it a popular choice in high-performance computing.

Let's look at a simple example in C that demonstrates OpenMP parallelization:

```
#include <stdlib.h>
#include <stdio.h>
#include <omp.h>

int kernel(float *input, float *output) {
    #pragma omp parallel
    {
        #pragma omp for
        for (int i = 0; i < 10; i++) {
```

```
            output[i] = input[i] * 2.0f;
        }
    }
}

int main() {
    float input[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    float output[10];
    kernel(input, output);
    for (int i = 0; i < 10; i++) {
        printf("%f ", output[i]);
    }
    return 0;
}
```

When compiled with OpenMP support (`clang -fopenmp`), this program will parallelize the loop across multiple threads. The same computation can be expressed in MLIR with the OpenMP dialect:

Parallel Regions (omp.parallel)

OpenMP organizes parallel execution through the concept of parallel regions. In MLIR, these are represented by the `omp.parallel` construct, which creates a team of threads that can execute code concurrently. This provides a more explicit way to control parallelization compared to the automatic parallelization we saw with the affine dialect.

The region is terminated with an `omp.terminator` operation, which is required for all OpenMP regions.

Worksharing Loops (omp.wsloop)

For parallel loops, the OpenMP dialect provides the `omp.wsloop` operation, which specifies that loop iterations will be executed in parallel by threads in the current context. Worksharing loops are typically used within parallel regions to distribute loop iterations across available threads:

```
omp.wsloop {
  omp.loop_nest (%i1, %i2) : index = (%c0, %c0) to (%c10, %c10) step (%c1, %c1) {
    %a = load %arrA[%i1, %i2] : memref<?x?xf32>
    %b = load %arrB[%i1, %i2] : memref<?x?xf32>
    %sum = arith.addf %a, %b : f32
    store %sum, %arrC[%i1, %i2] : memref<?x?xf32>
    omp.yield
  }
}
```

This example shows a nested loop being parallelized to add two matrices. The `omp.loop_nest` operation inside the `omp.wsloop` represents the nested loop structure with iteration variables, bounds, and steps.

Synchronization with Barriers (omp.barrier)

For thread synchronization, the OpenMP dialect provides the `omp.barrier` operation, which specifies an explicit barrier at the point where it appears. This ensures that all threads in a team reach the barrier before any thread proceeds:

```
// Code executed by all threads
omp.barrier
// Code executed after all threads reach the barrier
```

The `omp.barrier` operation is a simple operation without inputs or outputs, and its assembly format just includes the name and any attributes.

`omp` Dialect

Let's translate the above C code of a parallel loop to MLIR:

```
func.func private @kernel(%input: memref<10xf32>, %output: memref<10xf32>) {
  %loop_ub = llvm.mlir.constant(9 : i32) : i32
  %loop_lb = llvm.mlir.constant(0 : i32) : i32
  %loop_step = llvm.mlir.constant(1 : i32) : i32

  omp.parallel {
      omp.wsloop {
        omp.loop_nest (%i) : i32 = (%loop_lb) to (%loop_ub) inclusive step
(%loop_step) {
          %ix = arith.index_cast %i : i32 to index
          %input_val = memref.load %input[%ix] : memref<10xf32>
          %two = arith.constant 2.0 : f32
          %result = arith.mulf %input_val, %two : f32
          memref.store %result, %output[%ix] : memref<10xf32>
          omp.yield
        }
      }
    omp.barrier
    omp.terminator
  }

  return
}
```

And then we can have a main function that calls the kernel over a static defined input:

```
memref.global constant @input : memref<10xf32> = dense<[1.0, 2.0, 3.0, 4.0, 5.0,
6.0, 7.0, 8.0, 9.0, 10.0]>

// Define external references to printf and kernel
llvm.func @printf(!llvm.ptr, ...) -> i32
func.func private @kernel(%input: memref<10xf32>, %output: memref<10xf32>) -> ()
```

```
func.func @main() {
  %input = memref.get_global @input : memref<10xf32>
  %output = memref.alloc() : memref<10xf32>

  func.call @kernel(%input, %output) : (memref<10xf32>, memref<10xf32>) -> ()

  %lb = index.constant 0
  %ub = index.constant 10
  %step = index.constant 1

  %fs = llvm.mlir.addressof @fmt : !llvm.ptr

  scf.for %iv = %lb to %ub step %step {
    %el = memref.load %output[%iv] : memref<10xf32>
    llvm.call @printf(%fs, %el) vararg(!llvm.func<i32 (ptr, ...)>) : (!llvm.ptr,
f32) -> i32
  }

  return
}

// Define a constant string for the format specifier
llvm.mlir.global private constant @fmt("%f\0A\00") {addr_space = 0 : i32}
```

As an aside, variadic functions in MLIR allow for flexible argument passing, similar to C. A common pattern we'll use is calling functions like `printf`, which can take a variable number of arguments. In the main function, we're calling the `printf` function (which is variadic) to print the result. The following is equivalent to `printf("%f\n", el)` in C, demonstrating how to use `printf` in MLIR.

```
// Define external printf function from stdio.h
llvm.func @printf(!llvm.ptr, ...) -> i32

// Define a constant string for the format specifier
llvm.mlir.global private constant @fmt("%f\0A\00") {addr_space = 0 : i32}

// Call the variadic printf function with the format specifier and type
specialized arguments
%fs = llvm.mlir.addressof @fmt : !llvm.ptr
%el = llvm.mlir.constant(1.0 : f32) : f32
llvm.call @printf(%fs, %el) vararg(!llvm.func<i32 (ptr, ...)>) : (!llvm.ptr, f32)
-> i32
```

Here's the full example with the parallel loop and the main function:

```
memref.global constant @input : memref<10xf32> = dense<[1.0, 2.0, 3.0, 4.0, 5.0,
```

```
6.0, 7.0, 8.0, 9.0, 10.0]>

llvm.func @printf(!llvm.ptr, ...) -> i32
llvm.mlir.global private constant @fmt("%f\0A\00") {addr_space = 0 : i32}

func.func private @kernel(%input: memref<10xf32>, %output: memref<10xf32>) {
  %loop_ub = llvm.mlir.constant(9 : i32) : i32
  %loop_lb = llvm.mlir.constant(0 : i32) : i32
  %loop_step = llvm.mlir.constant(1 : i32) : i32

  omp.parallel {
      omp.wsloop {
        omp.loop_nest (%i) : i32 = (%loop_lb) to (%loop_ub) inclusive step
(%loop_step) {
          %ix = arith.index_cast %i : i32 to index
          %input_val = memref.load %input[%ix] : memref<10xf32>
          %two = arith.constant 2.0 : f32
          %result = arith.mulf %input_val, %two : f32
          memref.store %result, %output[%ix] : memref<10xf32>
          omp.yield
        }
      }
    omp.barrier
    omp.terminator
  }

  return
}

func.func private @main() {
  %input = memref.get_global @input : memref<10xf32>
  %output = memref.alloc() : memref<10xf32>

  call @kernel(%input, %output) : (memref<10xf32>, memref<10xf32>) -> ()

  %lb = index.constant 0
  %ub = index.constant 10
  %step = index.constant 1

  %fs = llvm.mlir.addressof @fmt : !llvm.ptr

  scf.for %iv = %lb to %ub step %step {
    %el = memref.load %output[%iv] : memref<10xf32>
```

```
    llvm.call @printf(%fs, %el) vararg(!llvm.func<i32 (ptr, ...)>) : (!llvm.ptr,
f32) -> i32
  }

  return
}
```

SCF to OpenMP Conversion Pass

The `-convert-scf-to-openmp` pass automatically converts parallel loops from the SCF dialect into equivalent OpenMP operations. This pass is particularly useful when you want to target OpenMP execution without manually writing OpenMP dialect code.

For example, an SCF parallel loop like this:

```
scf.parallel (%i) = (%c0) to (%c100) step (%c1) {
  %val = memref.load %A[%i] : memref<100xf32>
  %result = arith.addf %val, %val : f32
  memref.store %result, %B[%i] : memref<100xf32>
  scf.yield
}
```

Will be converted to OpenMP operations:

```
omp.parallel {
  omp.wsloop for (%i) : i32 = (%c0) to (%c100) step (%c1) {
    %val = memref.load %A[%i] : memref<100xf32>
    %result = arith.addf %val, %val : f32
    memref.store %result, %B[%i] : memref<100xf32>
    omp.yield
  }
  omp.terminator
}
```

The pass handles several important conversions, including the transformation of `scf.parallel` operations into `omp.parallel` regions that contain `omp.wsloop`, while preserving loop bounds and step sizes. Additionally, reduction operations in SCF are mapped to OpenMP reductions, and nested parallel loops are properly managed. This conversion is typically utilized as part of a larger lowering pipeline, which is often followed by `-convert-openmp-to-llvm` to generate LLVM IR that can be executed with OpenMP runtime support.

External Resources

MLIR: Affine Dialect

MLIR: Polyhedral Structures

MLIR: OpenMP Dialect

RFC: OpenMP dialect in MLIR

OpenMP: OpenMP 5.0 Specification

Parallelizing Applications with MLIR

OpenMP in Flang using MLIR

OpenMPOps.td

OpenMPToLLVMIRTranslation.cpp

Extending Polygeist to Generate OpenMP SIMD and GPU MLIR Code

OpenMP in a Nutshell

MLIR Tutorial: Building a Compiler with MLIR

2023 EuroLLVM - MLIR Dialect Design and Composition for Front-End Compilers

Add conversion from SCF parallel loops to OpenMP

Open MLIR Meeting 2-15-2024: OpenMP GPU Target Offloading

Mark parallel regions as allocation scopes